

1 Expresie – Megoldás

1.1 A probléma áttekintése

Adott n darab számjegy ($0 \dots 9$), amelyeket p darab $+$ és q darab $-$ műveleti jel segítségével kell összekapcsolnunk. A p és q jelek $p + q$ darab osztópontot hoznak létre, így pontosan $lg = n - p - q$ darab számjegy marad egybefüggő részsorozatban (azaz ezeket a számjegyeket semmi sem választja el egymástól). A cél az expression értékének maximalizálása.

1.2 Alapvető megfigyelések

1. A végső eredmény felépítése: a kifejezés értéke egyetlen nagy $p + q + 1$ bites szám lesz, amely a egybefüggő részsorozatból és az előtte/utána elhelyezett számjegyekből tevődik össze.
2. A részsorozat hossza fix: $lg = n - p - q$. Ezt a sorozatot mindenképpen egybefüggően kell hagynunk, mert különben túllépnénk a rendelkezésre álló jelek számán.
3. A p darab $+$ és q darab $-$ összesen $p + q$ számjegyet helyez el a részsorozat *kívül*.

1.3 A kulcs ötlet

Az eredményt úgy maximalizáljuk, ha a **lexikografikusan legnagyobb** lg hosszúságú részsorozatot tartjuk meg egybefüggően, és az összes többi számjegyet a lehető legkedvezőbben helyezzük el előtte vagy utána.

Miért működik ez? A legnagyobb helyi értékű számjegyeket szeretnénk a kifejezés elejéhez minél közelebb vinni. Ha egy lg hosszú részsorozatot nem bontunk szét, akkor az egész egyetlen számmá alakul, ahol az elején álló számjegyek dominálnak a végi számjegyek felett. Minél nagyobb ez a kezdőszakasz, annál nagyobb a teljes szám.

A megmaradt $p + q$ számjegyet **növekvő sorrendbe** rendezzük. Ekkor:

- Az első q darab (a legkisebbek) a $-$ műveleti jelek mögé kerülnek — így a lehető legkisebb értéket vonjuk ki.
- A maradék p darab (a legnagyobbak) a $+$ műveleti jelek mögé kerülnek — így a lehető legnagyobb értéket adjuk hozzá.

Ha a részsorozat nem az elején kezdődik, akkor az első számjegyet ($s[0]$) $+$ előjellel tesszük a részsorozat *elejére*. Egyébként a részsorozat a kifejezés elején van, és nincs szükség extra előjeles tagra.

Ez az eljárás biztosítja, hogy az eredmény maximális legyen.

1.4 Az algoritmus lépései

1. Beolvassuk n , p , q és a s karaktersorozatot.
2. Kiszámítjuk $lg = n - p - q$.
3. Megkeressük a lg hosszúságú, lexikografikusan legnagyobb részsorozat kezdőpozícióját (**imax**).
4. Ha **imax** > 0, akkor a **s**[0] számjegyet + előjellel hozzáadjuk az összeghez.
5. Összegezzük a **imax** pozíciót **nem** tartalmazó összes számjegyet egy a tömbbe, majd **növekvő sorrendbe** rendezzük.
6. Az a tömb első q elemét kivonjuk, a többit hozzáadjuk az összeghez.
7. A lg hosszú maximális részsorozatot hozzáadjuk (vagy kivonjuk) az összegből a végső eredményből.
8. Kiírjuk az eredményt.

1.5 Implementációs részletek

A számításokat kézzel végzett nagy számokkal végezzük, mivel az eredmény akár n számjegyű is lehet. Az **adun** és **dif** eljárások a decimális összeadást és kivonást implementálják.

Az **comparnr** és **compars** függvények a nagy számok és részsorozatok lexikografikus összehasonlítására szolgálnak.

A fő ciklus a lexikografikusan legnagyobb részsorozat keresésére:

```
imax=0;
if (p>0)
    {for (i=1; i<n-lg+1; i++)
        if (compars(i,imax)>0) imax=i;
    }
```

A $p > 0$ ellenőrzés biztosítja, hogy csak akkor keresünk, ha van legalább egy + jel — egyébként a teljes szám marad egybefüggő.

A rendezés és az összeg kiszámítása:

```
sort(a,a+nr);
for (i=0; i<q; i++) suma=suma-(a[i]-'0');
for (i=q; i<nr; i++) suma=suma+(a[i]-'0');
```

A **suma** változó a kivonandó és összeadandó számjegyek előjeles összege, amelyet később hozzáadunk a lexikografikusan legnagyobb részsorozathoz.

1.6 Idő- és tárkomplexitás

- A lexikografikus maximum keresése: $O(n \cdot lg) = O(n^2)$ a triviális implementációval, de a gyakorlatban $lg \leq n$, így elfogadható.
- A rendezés: $O((p + q) \log(p + q))$, ami $O(n \log n)$.
- Az összeadás/kivonás: $O(lg + \log_{10}(p+q))$, azaz lineáris a számok hosszában.

Összességében a megoldás $O(n \log n)$ idő- és $O(n)$ tárkomplexitású, ami teljes mértékben megfelel a korlátoknak.